

TECHNICAL & EXECUTIVE REPORT

Cortex

The Five-Signal Malware Detection Pipeline Architecture, Evidence, Limitations, and Roadmap

Prepared: 2026-08-27

System: `cortex-ml` — independent clean reproduction, five detection signals

Sources: repository `PROJECT_HISTORY_REPORT.md`, `README.md`, and source at commit `9331159` plus the later uncommitted `signify` fix & documentation updates

Audience: mixed — every major section opens with a plain-language summary, followed immediately by full technical depth

All figures in this document are taken directly from the repository's own records and code. No number has been invented and no documented limitation has been softened.

CONTENTS

Table of Contents

- 1** Executive Summary
- 2** System Architecture — What Happens When a File Comes In
- 3** Per-Signal Deep Dive
 - 3.1 Static · 3.2 Behavioral · 3.3 Memory · 3.4 Network · 3.5 Emulation
- 4** Policy Engine / Decision Logic
- 5** Known Limitations and Risk Mitigations
- 6** Engineering Discipline
- 7** Current Status and Next Steps
- 8** Appendix — Full Metrics, Thresholds, Splits, Config

Executive Summary

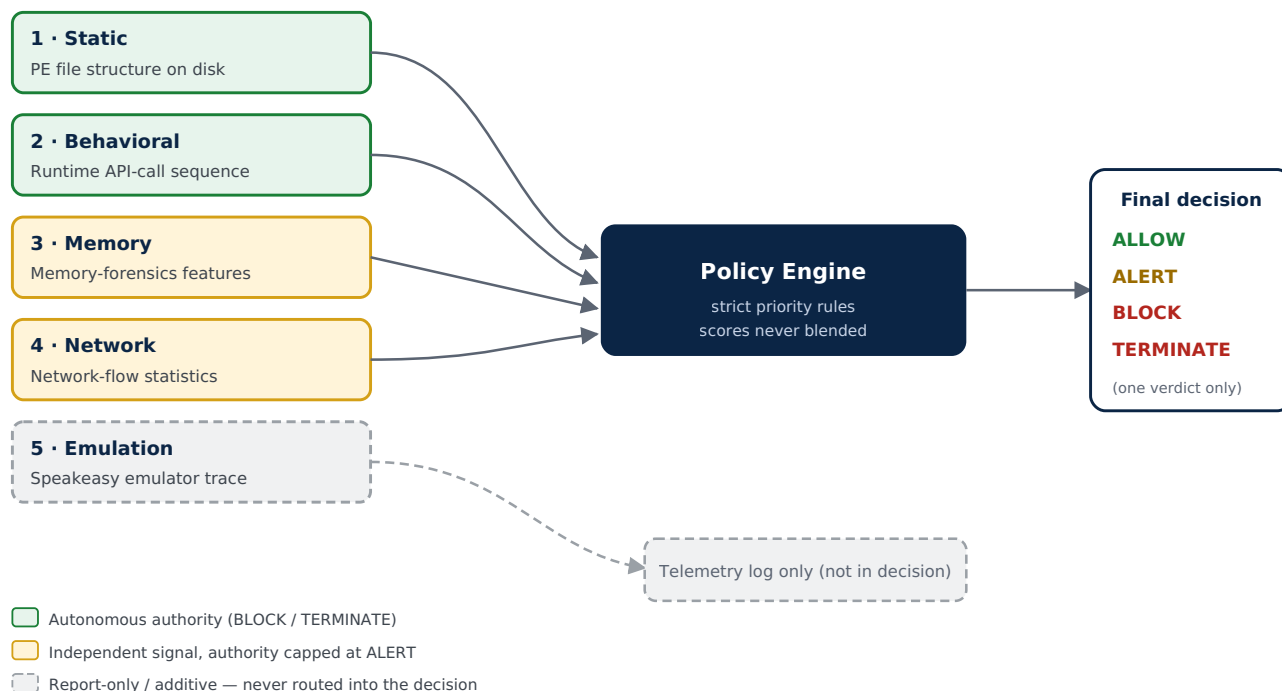
IN PLAIN TERMS

Cortex is a Windows malware detector that looks at a suspicious program in five independent ways instead of relying on a single test. Each of the five "signals" examines a different kind of evidence: the file's structure on disk, the sequence of system actions it performs, what it looks like in a computer's memory, the network traffic it generates, and how it behaves inside a safe emulator. The five results are then combined by a **rule-based policy engine** that follows a strict priority order — it never averages scores together.

Why five and not one? Because every single detection method has a blind spot. A file that is disguised on disk may still act maliciously once running. Fileless malware never writes a file to disk at all, so only the memory signal can see it. Slow, quiet intrusions hide from network analysis but show up in system-call behavior. Using five overlapping signals means one signal's weakness is covered by another's strength.

Headline results: the two most-trusted signals (Static and Behavioral) score AUC 0.997–0.999 across three separate implementations and have been checked against real malware and real signed Microsoft tools. Two newer signals (Memory and Network) are fully trained with strong lab metrics but are deliberately held back from the most severe automated actions until they are validated on real-world data. The fifth signal (Emulation) is trained but kept out of decisions entirely because testing exposed a serious accuracy collapse over time.

Five independent signals



High-level architecture. Five signals feed a rule-priority policy engine that emits exactly one verdict. Static and Behavioral can act autonomously; Memory and Network are capped at ALERT; Emulation is logged but never decides.

TECHNICAL DETAIL

What Cortex is. `cortex-ml` is a from-scratch, independently engineered reproduction of a malware-detection architecture previously proven across two earlier repositories (`malware-ml`, the research phase, and `cortex-endpoint`, a packaged Windows executable). The prior work hit AUC > 0.998 static and AUC > 0.99 behavioral and passed real-world validation against live malware samples and signed system tools. `cortex-ml` reproduces that detection logic in a footprint small enough to hand to another team, began with two signals (Static, Behavioral), and has since been extended to five.

Why five independent signals. The architecture contract (`Architecture.txt`) specifies sequential gating with no ensembling: signals are combined by fixed-priority verdict rules, never by averaging scores. Each signal exists because it sees a threat class the others structurally cannot:

- **Static** — catches known-bad file structure before anything executes, but is blind to a clean-looking file that turns malicious at runtime.
- **Behavioral** — catches malicious runtime API-call patterns, but needs an execution trace to exist.
- **Memory** — catches injected code and fileless malware that never write a file to disk, so there is nothing for Static to score.
- **Network** — catches malicious traffic patterns (DDoS, brute force, botnet C2) from flow statistics alone.
- **Emulation** — catches malicious behavior inside a Speakeasy emulator before the sample ever runs on a real host.

Headline results (held-out test sets, current trained models).

Signal	Key metric	Value	Authority
Static	AUC-ROC / detection rate / FPR @ 0.5	0.9988 / 98.17% / 1.10%	BLOCK (autonomous)
Behavioral	Accuracy / precision / recall / AUC-ROC	98.36% / 100% / 98.08% / 0.9973	TERMINATE (autonomous)
Memory	AUC-ROC / precision / recall / FPR	1.0 / 0.9891 / 1.0 / 1.13%	ALERT (capped)
Network	AUC-ROC / precision / recall / FPR	0.9972 / 0.9969 / 0.9752 / 0.098%	ALERT (capped)
Emulation	Test AUC-ROC / pooled detection @ 1% FPR	0.8737 / ~49.9%	Report-only

WHAT IS FULLY TRUSTED VS. STILL BEING VALIDATED — ONE HONEST PARAGRAPH

Static (BLOCK) and Behavioral (TERMINATE) carry full autonomous authority: both are validated across three independent implementations with consistently strong AUCs, and Static passed real-world checks against actual malware and signed Microsoft tools. Memory and Network are real, fully-trained, ONNX-exported signals with excellent held-out metrics, but each is **deliberately capped at ALERT** for its own specific, diagnosed reason — Memory because its benign training data all comes from a single baseline virtual machine (22 of 62 features separate the classes almost perfectly, which looks more like "recognising that one VM" than "recognising malice"); Network because it detects only 10.75% of slow "Infiltration" attacks and because models that look near-perfect on its training dataset are documented to collapse toward random on real traffic. Emulation is trained but held **report-only**: an ablation confirmed its malware recall falls from ~71% to ~42% between the January and April 2022 collection eras and that shrinking the model does not fix it, and it beats a trivial lookup-table baseline by only ~3-4 percentage points. Separately, a recently found static feature-extraction bug (a broken optional `signify` import that silently zeroed one feature group) has been partially fixed, but the residual feature-fidelity gap versus the reference extractor and the absence of an automated parity test remain the top open correctness item for the static path.

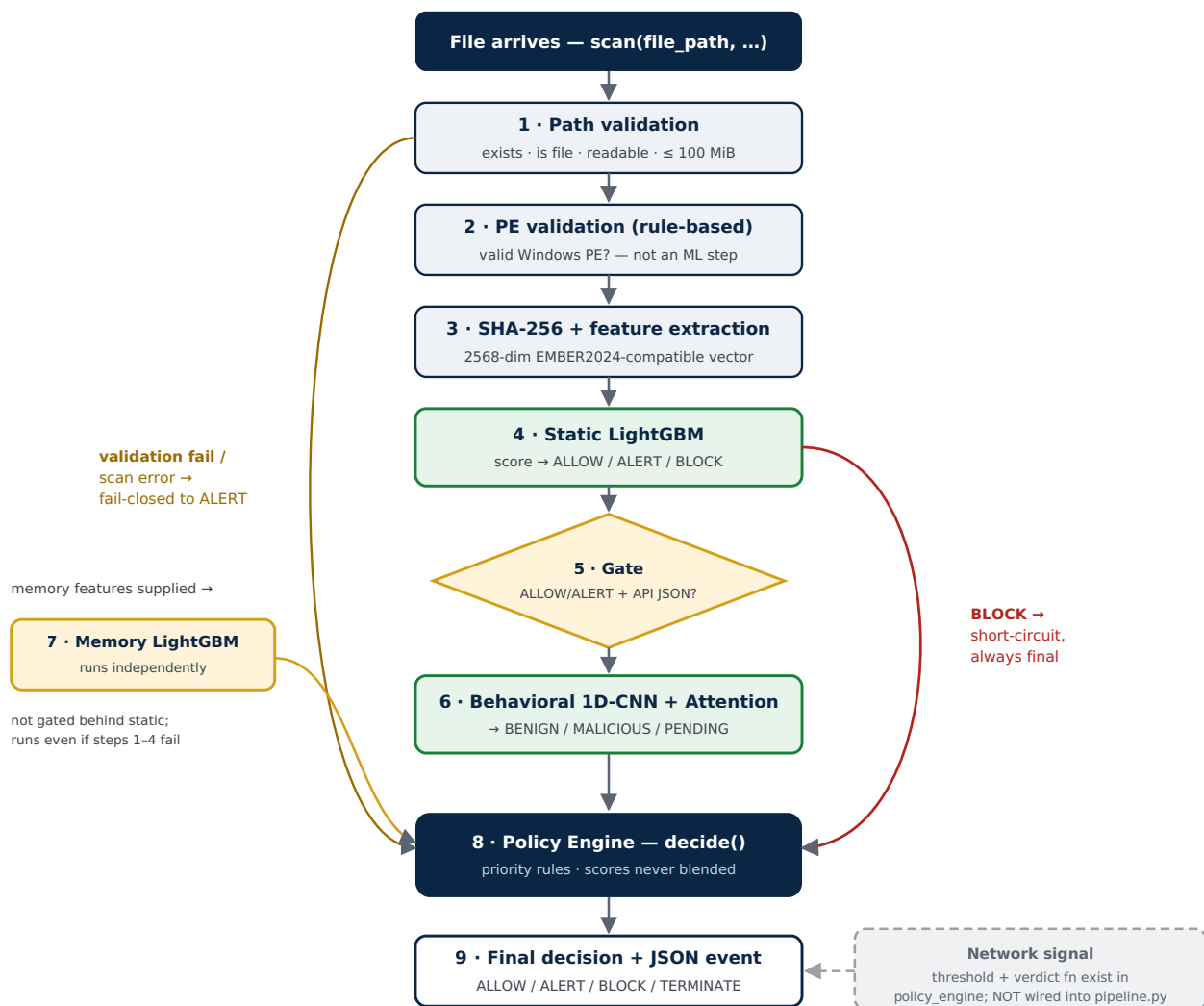
SECTION 2

System Architecture — What Happens When a File Comes In

IN PLAIN TERMS

When a file arrives, Cortex runs a fixed sequence of checks. First it makes sure the file is real, readable, not too large, and actually a Windows program. Then it converts the file into a long list of numeric measurements and asks the Static model for a verdict: **allow**, **alert**, or **block**. A **block** is final and everything stops. An **allow** or **alert** lets the file move on to the Behavioral model, if a record of the program's actions is available.

The Memory check is different: it runs **on its own, in parallel**, and is not made to wait behind the file checks. That is deliberate — fileless malware never creates a file on disk, so if Memory had to wait for a valid file to be found, it would never get to examine exactly the threat it exists to catch. The Network check is defined in the decision rules but is **not yet connected** into the scan pipeline itself; this report says so plainly rather than implying it is live. Finally, every result is passed to the policy engine, which applies its priority rules and produces one final decision plus a structured JSON event record.



The actual scan flow in `inference/pipeline.py`. Solid arrows are the main spine; the red path is the unconditional BLOCK short-circuit; the amber lane is Memory running independently; the dashed box is Network, defined in the policy engine but not yet called by the pipeline.

STEP-BY-STEP WALKTHROUGH (`CORTEXPIPELINE.SCAN()`)

- Memory first, deliberately (step 7 in code order).** If a memory feature vector is supplied, Cortex-Memory runs before any file validation and independently of it. This is intentional: a fileless / injected threat has no on-disk file to resolve, so gating Memory behind path or PE validation would disable it for exactly the threat class it exists to catch.
- Path validation.** `_validate_path()` checks the path exists, is a regular file, is readable, and is ≤ 100 MiB (`MAX_FILE_SIZE_BYTES`). On failure the static verdict becomes `ERROR`, a reason code is recorded, and the policy engine is called immediately with whatever verdicts exist (Memory may still have produced one).
- Rule-based PE validation.** `feature_extractor.is_valid_pe(bytez)` — a structural check, not an ML prediction. A non-PE or corrupt file yields `static_verdict = ERROR` and reason code `invalid_or_non_pe_file`, then straight to the policy engine.

4. **SHA-256 + feature extraction.** The file's SHA-256 is recorded and `PEFeatureExtractor.feature_vector()` produces the 2568-dimensional EMBER2024-compatible vector (12 feature groups; see §3.1).
5. **Static model.** `static_model.predict_proba()` returns a single Platt-calibrated malicious probability. `static_verdict_from_score()` maps it: `< 0.6163460957` → ALLOW; `< 0.9950119117` → ALERT; otherwise → BLOCK. Any exception here is caught and becomes `static_verdict = ERROR` with reason `static_scan_exception` — it is never allowed to crash the scan or silently pass.
6. **Gate into Behavioral.** If `static_verdict == ALLOW` and an API-call JSON path was supplied, Behavioral runs. If `static_verdict == ALERT` and API-call JSON was supplied, Behavioral also runs (an ALERT can still be upgraded to TERMINATE by a MALICIOUS behavioral verdict). If `static_verdict == BLOCK`, Behavioral is never run — the block is already protected and final. The tokenizer returns `PENDING` for empty / insufficient traces rather than guessing.
7. **Memory in parallel (already computed in step 1).** Its score and verdict sit alongside the static and behavioral verdicts with no dependency between them.
8. **Network — current wiring status, stated honestly.** `NETWORK_MALICIOUS_MIN`, `NetworkVerdict`, and `network_verdict_from_score()` all exist in `inference/policy_engine.py`, and `decide()` accepts a `network_verdict` argument. But `CortexPipeline.scan()` has no network parameter and never computes or passes a network verdict — the pipeline was never extended. Network is a trained, ONNX-exported model with a derived threshold; it is simply not yet called end-to-end.
9. **Policy engine.** `decide(static_verdict, behavioral_verdict, memory_verdict[, network_verdict])` applies the priority chain in §4 and returns a `FinalDecision` plus reason codes.
10. **Structured output.** `to_security_event()` wraps the result as a JSON envelope with a UTC timestamp and a UUID correlation ID: per-signal `{score, verdict}` blocks, the final decision, and the accumulated reason codes.

WHY SOME SIGNALS GATE ON OTHERS AND SOME DO NOT

Static → Behavioral is gated because Behavioral analysis exists to refine an uncertain static result. A static BLOCK is already the most severe file-level verdict and is unconditionally final, so running Behavioral after it would add cost and risk (a MALICIOUS→TERMINATE upgrade is possible but a BLOCK already stops the file). An ALLOW or ALERT is where a second opinion has value, so the gate opens there.

Memory does not gate because its target — injected code and fileless malware — by definition leaves no valid on-disk PE for steps 1–4 to evaluate. If Memory waited behind static validation, a fileless intrusion that fails PE validation (there is no PE) would never be examined. So Memory runs whenever memory features are supplied, regardless of the static verdict and even when path/PE validation failed outright.

Network, once wired, is designed to be independent for the same reason: network flows exist whether or not a file was submitted for scanning, and the threats it targets (C2 beaconing, exfiltration, floods) are not tied to a single on-disk artifact.

Fail-closed everywhere. Every validation failure and every caught scan exception resolves to `ERROR`, and `decide()` maps every `ERROR` to ALERT — never to a silent ALLOW.

SECTION 3

Per-Signal Deep Dive

Each subsection follows the same shape: a plain-language summary, then what the signal detects and why it is separate, the real dataset and its provenance/licensing, the architecture chosen and why (including where it deliberately diverged from a reference design and the evidence for that), how the threshold was derived, and the current measured performance.

3.1 Cortex-Static

IN PLAIN TERMS

Static analysis looks at a Windows program **without running it**. It measures thousands of structural properties of the file — its headers, sections, imported functions, embedded strings, byte patterns, digital signature — and a gradient-boosted decision-tree model decides how malicious that structure looks. This is the fastest signal and the only one that can act with full authority: a Static **BLOCK** is always final. It is trained on EMBER2024, a large public research dataset of labelled Windows programs.

WHAT IT DETECTS AND WHY IT IS A SEPARATE SIGNAL

Malicious PE file structure — packers, anomalous headers, suspicious import tables, known-bad string/IOC patterns, signature anomalies — detectable before any execution. It is the first line of defence and the only signal with no execution-trace dependency, so it is the natural place for the highest-authority verdict.

DATASET & PROVENANCE

EMBER2024, streamed from HuggingFace (`FutureComputing4AI/EMBER2024` -derived), **Apache-2.0**, PE-format records only. `datasets.load_dataset()` hit an Arrow schema-inference crash on this dataset, so `data/download_ember2024.py` streams the PE records directly from the HF zips and deduplicates on `sha256` in bounded batches. The raw download contained every row twice (~50% duplicate rate, the signature of "every sample present twice"): **4,680,000 raw train rows → 2,340,000 unique; 1,080,000 raw test rows → 539,940 unique**. **Leakage check:** zero train/test hash overlap after the dedup fix (a handful of within-test-shard repeats noted, not train/test crossing).

ARCHITECTURE CHOSEN, AND WHY

LightGBM gradient-boosted trees over the 2568-dim feature vector, with a Platt calibrator (`LogisticRegression`) fitted on top of the raw booster. Hyperparameters: `num_leaves=255` , `max_depth=15` , `learning_rate=0.03` , `min_child_samples=50` , `is_unbalance=True` . Training used 2,996 boosting rounds, ~2.1h wall time. Trees are the established architecture for EMBER-family tabular features and are unaffected by input scaling.

Divergence from the reference, and the evidence for it. Two deliberate departures:

- **Feature extractor rewritten for formula-fidelity, not copied.** `features/pe_features.py` matches the real EMBER2024 / "thrember" reference extractor's group budget and formulas exactly (fixed string-regex vocabulary, header categorical dictionaries, section stats, data-directory layout, hashing schemes) — replacing an earlier version built from independently-guessed field semantics. The reason: a live PE-byte vector and an EMBER2024 training record must be numerically column-compatible, or the model learns one semantics at training time and is fed another at inference — a skew that does not show up in offline eval. Every field inside each group was still chosen independently; this is a formula-compatible reimplementaion, not a line-for-line port.
- **Memory pressure solved with OS swap, not data subsampling.** Training on the full 2.34M-row / 2568-feature split repeatedly OOM-killed a 38 GB machine. Fixes applied in order: a streaming train/val loader instead of a full fancy-indexed DataFrame, `free_raw_data=True` + explicit `del / gc.collect()`, reduced `n_jobs`, a capped `histogram_pool_size`, two-round Dataset construction — and finally **adding swap space at the system level**. The earlier research repo, under the same class of memory pressure, instead trained on a 51.3% subsample; a separate attempt there at sequential chunked training was explicitly rejected after validation TPR degraded across chunks (0.917 → 0.171 → 0.174). `cortex-ml` trained on 100% of the deduplicated data.

THRESHOLD DERIVATION

`STATIC_ALLOW_MAX` and `STATIC_BLOCK_MIN` were re-derived from scratch against `cortex-ml`'s own trained model — `models.static_lgbm.find_threshold_for_fpr()` run on the calibrated probabilities over the 539,940-row deduplicated EMBER2024 test split, not carried over from the prior project's placeholder values.

Boundary	Constant	Value	Target FPR	Actual FPR	Detection @ boundary
score < this → ALLOW	<code>STATIC_ALLOW_MAX</code>	0.6163460957	0.01	0.009999	0.9803
score ≥ this → BLOCK	<code>STATIC_BLOCK_MIN</code>	0.9950119117	0.001	0.000997	0.9168

The BLOCK boundary hits the architecture doc's max-FPR-at-0.1% bar. The ALLOW boundary uses target FPR 0.01 because the score distribution has a cliff between target FPR 0.005 and 0.01 (threshold drops from ~0.96 to ~0.62): 0.01 is where ALLOW stops sitting in near-BLOCK territory and meaningfully separates from ALERT. Scores between the two boundaries → ALERT.

CURRENT PERFORMANCE

Held-out test (deduplicated EMBER2024 test split): **AUC-ROC 0.9988, detection rate 98.17%, FPR 1.10%** at threshold 0.5 (a less conservative operating point than the deployed ALLOW/BLOCK boundaries). **ONNX export:** `cortex_static.lgbm` (78.8 MB) → `cortex_static.onnx` (61.4 MB) with the Platt calibrator merged into the same graph; verified against 10,000 real test rows — mean abs diff ≈ 0, max abs diff 0.0032 (float noise), **zero**

verdict differences at the production thresholds. No INT8 quantization (tree ensembles are unaffected by weight quantization).

KNOWN LIMITATION

A silent `signify 0.9.2` import failure had zeroed the entire 8-dim `authenticode` feature group at inference (see §5.4 and §6). It is now ported to the 0.9.x API, but a residual feature-fidelity gap versus the thrember reference remains — `cortex-ml`'s vector is systematically shifted toward "malicious" — and there is still **no automated feature-parity test**. The earlier research-repo candidate also showed a PyInstaller-build-artifact false positive on a signed benign binary; this has not been re-tested against `cortex-ml`'s own static model.

3.2 Cortex-Behavioral

IN PLAIN TERMS

Behavioral analysis looks at **what a program actually does** — the ordered list of Windows system functions it calls while running. A neural network reads that sequence and decides whether the pattern of actions looks malicious. This catches malware that is disguised well enough on disk to slip past Static but gives itself away through its behavior. A Behavioral **MALICIOUS** verdict can autonomously trigger the most severe action, **TERMINATE**.

WHAT IT DETECTS AND WHY IT IS A SEPARATE SIGNAL

Malicious runtime intent expressed as an API-call sequence (e.g. injection, persistence, C2 setup, credential access). It exists separately because a structurally clean file can still behave maliciously, and because a sequence model captures ordering and context that static structural features cannot.

DATASET & PROVENANCE — THREE INDEPENDENTLY-VERIFIED SOURCES

Source	Origin / licence context	Classes	Rows
Mal-API-2019	GitHub <code>ocatak/malware_api_class</code>	malicious only	7,107
MalbehavD-V1	GitHub <code>mpasco/MalbehavD-V1</code>	both	2,570
Carpenter <code>benign.json</code>	Kaggle (benign-only subset used deliberately)	benign only	101

9,778 combined records before dedup. Carpenter's malicious batches are deliberately never downloaded, to avoid unverified malicious labels. All API names are lowercased into one shared vocabulary space (>96% clean casing-only overlap between Mal-API-2019 / MalbehavD-V1; >67% between Carpenter and the other two). Deduplication is on record `id`, **not** on the `api_calls` sequence — MalbehavD-V1's shorter sequences legitimately recur across genuinely different real files, so sequence-based dedup would have wrongly discarded ~1,400 distinct samples.

Leakage check: a naive row-independent split left **85 sequences spanning multiple splits**. `scripts/split_behavioral.py` now does a grouped stratified 80/10/10 split by `(source, label)` with every row sharing an identical `api_calls` sequence forced into the same split; verified 0 sequences span more than one split after the fix (a hard invariant).

ARCHITECTURE CHOSEN, AND WHY

1D-CNN + multi-head self-attention, single sigmoid score (`models/behavioral_cnn.py`): token embedding + learned positional embedding → conv stack (channels 128→256→128→128, kernel sizes 3,5,7,3, each with BatchNorm + dropout 0.3) → multi-head self-attention (4 heads, residual + LayerNorm) → global pool → linear sigmoid head. Fixed sequence length 100, `<PAD>` / `<UNK>` tokens. Raw sigmoid output, no calibration layer. `embed_dim` defaults to 128 in the training CLI (dropping to 64 is a documented option). This follows the architecture brief's CNN + attention shape.

Divergence from the reference: the earlier research-repo behavioral model (Model A, Oliveira-sourced) used embedding → 2×Conv1D → global max pool → sigmoid with no attention. `cortex-ml` added the self-attention block and moved to the three-source Mal-API-2019 + MalbehavD-V1 + Carpenter corpus. The evidence that drove a more conservative overall approach is Model A's threshold-generalization failure (below).

THRESHOLD DERIVATION

`BEHAVIORAL_MALICIOUS_MIN = 0.60`. Derived by a threshold sweep (0.10–0.99) over the val+test splits combined (1,835 rows, only 274 benign) — deliberately **not** `find_threshold_for_fpr()` at a precise target, because 274 benign samples (~0.36% resolution each) cannot support a precise FPR claim. Exactly one benign sample is a persistent false positive at every threshold below ~0.922 — a MalbehavD-V1 trace with `setsockopt` / `ioctlsocket` / `wsastartup` / `getsockname` alongside routine registry/system calls, plausibly confusable with C2 setup; the model scores it 0.921006, confidently wrong. The threshold was **deliberately not raised** to clear this single case: $n=1$ is not a sound basis for a permanent recall trade-off across the whole malicious population, and Behavioral has no rule-based backstop in this repo yet. Malicious false-negative count on val+test climbs slowly as the threshold drops (57 at 0.95, 49 at 0.90, ~38 at 0.60, 36 at 0.50).

CURRENT PERFORMANCE

Held-out test at threshold 0.60: **accuracy 98.36%, precision 100%, recall 98.08%, AUC-ROC 0.9973**. 18 epochs, early-stopped, ~4 minutes wall time on CPU. **ONNX export:** two bugs found and fixed — `torch.onnx.export()` defaulting to the dynamo exporter (needs `onnxscript`, not a dependency; forced legacy `dynamo=False`), and `quantize_dynamic()` trying to quantize the Conv1d stack (no `ConvInteger` CPU kernel; INT8 quantization restricted to `MatMul` / `Gemm`).

KNOWN LIMITATION

The single hard benign false positive above; no rule-based overlay for high-risk call combinations independent of the ML score (flagged as the targeted future fix, not blanket threshold tuning). And unlike the prior research-repo model, `cortex-ml`'s

behavioral threshold **has not yet been stress-tested** against a genuinely held-out generalization check.

3.3 Cortex-Memory

IN PLAIN TERMS

Memory analysis examines a snapshot of a computer's RAM for the fingerprints of malicious activity — code injected into another process, hidden modules, unusual handle or driver counts. Crucially, this works **even when malware never writes a file to disk**. Because of that, the Memory check runs on its own and is not made to wait for the file checks. Its metrics on the research dataset are near-perfect — which is itself a reason for caution, so its authority is capped at **ALERT**.

WHAT IT DETECTS AND WHY IT IS A SEPARATE SIGNAL

Injected / fileless malicious activity visible only in a memory image: process hollowing, injected regions (`malfind`), hidden processes (`psxview`), anomalous handle/DLL/callback/driver statistics. It is independent because there is often no on-disk file for any other signal to score.

DATASET & PROVENANCE

CIC-MalMem-2022 (Carrier et al., ICISSP 2022) — a public research dataset, no licensing ambiguity. The official UNB/CIC portal requires a manual download form and, at pull time (2026-08-25), was missing 3 of the dataset's 4 top-level categories; the working copy was taken from the Kaggle mirror `luccagodoy/obfuscated-malware-memory-2022-cic`. 58,596 rows, 55 VolMemLyzer-derived numeric memory-forensics features plus `Category` (family + `Benign`) and `Class` (`Benign` / `Malware`). Schema validated **structurally against the real file** (column count, dtypes, exact `Class` value set, first/last column identity), not trusted from documentation.

ARCHITECTURE CHOSEN, AND WHY

LightGBM + Platt calibration, mirroring `models/static_lgbm.py`'s shape exactly. Inputs: the 55 raw VolMemLyzer columns plus **7 derived ratio/aggregate features** (62 total) — `callbacks_anonymous_ratio`, `svcsan_driver_ratio`, `handles_file_ratio`, `handles_mutant_ratio`, `psxview_hiding_score`, `ldrmodules_hidden_ratio`, `malfind_injection_rate` — each tied to a specific memory-forensics indicator. Hyperparameters: `num_leaves=31`, `max_depth=-1`, `learning_rate=0.05`, `min_child_samples=20`; `is_unbalance` deliberately omitted (the split is near 50/50, unlike Static). Best iteration 71 (val-driven early stopping).

Divergence / deliberate design choices: a `MemoryFeatureScaler` (train-only-fit standardisation) is built and tested but **not applied** — tree-ensemble splits are provably invariant to any monotonic per-feature transform, so scaling every value would change zero split decisions while adding a second fitted artifact to carry into the ONNX export. Handles-per-process and DLL-per-process ratios were deliberately **not** added as derived features because VolMemLyzer already provides them directly. The scaler stays available for a future non-tree Cortex-Memory variant.

THRESHOLD DERIVATION

`MEMORY_MALICIOUS_MIN = 0.0005358335957155212`, via `models.memory_lgbm.find_threshold_for_fpr()` at **target FPR 0.01** against calibrated val+test-combined probabilities (11,860 rows, **5,860 benign** — far more statistical power than Behavioral's 274; one false positive moves observed FPR by ~0.017%). Sweep:

Target FPR	Threshold	Actual FPR	Benign FPs	Detection
0.10%	0.100225	0.085%	5 / 5,860	99.95%
0.50%	0.000627	0.478%	28 / 5,860	100.00%
1.00%	0.000536	0.939%	55 / 5,860	100.00%
2.00%	0.000502	1.724%	101 / 5,860	100.00%
5.00%	0.000446	4.744%	278 / 5,860	100.00%

1% chosen: the best-supported target (55 observed FPs vs only 5 at 0.1%, too few to trust) that costs nothing in recall — detection is already 100% at this and every looser target.

CURRENT PERFORMANCE

Held-out test at this threshold: **AUC-ROC 1.0, precision 0.9891, recall 1.0, F1 0.9945, FPR 1.13%** (33/2,930 benign test rows). **ONNX export:** verified against the real model and the actual 5,930-row test split — mean abs error 6.7e-9, max abs error 2.0e-6, **zero prediction mismatches** at the operating threshold across all test rows.

KNOWN LIMITATION — THE REASON FOR THE ALERT CAP

22 of the 62 features individually exceed 0.95 AUC, confirmed on both the train split and the untouched test split — e.g. `handles.avg_handles_per_proc` is a tight 208–318 band for benign (std 17.5) but ranges 71–33,784 for malware; `dlllist.avg_dlls_per_proc` is 34.5–53.2 for benign but drops to 7.3 for malware. This pervasive multi-feature separability on data the model never trained on rules out our own split leaking and points at the dataset itself: **every CIC-MalMem-2022 benign sample is a repeated capture of "normal user behavior" on a single baseline Windows 10 VM**, while malicious samples span far more varied executions. The model may be learning "does this look like that one baseline VM" more than "is malicious behavior present." This is a specific reason to keep the ALERT cap, not a generic hedge. Also unsolved: benign near-duplicates (only exact duplicates are merged), because nothing in the schema identifies which benign rows share a capture session.

3.4 Cortex-Network

IN PLAIN TERMS

Network analysis looks only at **statistics about network connections** — how long they last, how many packets and bytes flow each way, timing between packets — never the packet contents. From those numbers a decision-tree model flags traffic that

matches attack patterns like denial-of-service, brute-force login attempts, or botnet command-and-control. It is trained on a large public intrusion-detection dataset. It has a real blind spot for slow, quiet intrusions, and near-perfect scores on this kind of dataset are known to collapse on real traffic — so its authority is capped at **ALERT**. The model is trained and exported but **not yet wired into the scan pipeline**.

WHAT IT DETECTS AND WHY IT IS A SEPARATE SIGNAL

Malicious traffic patterns from CICFlowMeter flow-statistics: DDoS (HOIC/LOIC), brute force (FTP/SSH/web), botnet, DoS (Hulk/GoldenEye/Slowloris/SlowHTTPTest). Independent of any scanned file — flows exist on their own.

DATASET & PROVENANCE

CSE-CIC-IDS2018 (Sharafaldin et al., CIC), **chosen over UNSW-NB15 specifically because UNSW-NB15 has a documented licensing ambiguity** while CSE-CIC-IDS2018 has an explicit, clear redistribution licence. Hosted on AWS Open Data (`s3://cse-cic-ids2018/` , `ca-central-1`) with no account or access form. Only the `Processed Traffic Data for ML Algorithms/` prefix was synced (10 CICFlowMeter CSVs, 6.4 GiB) — not the raw PCAPs. 16,233,002 raw rows across 10 capture days (Feb–Mar 2018).

ARCHITECTURE CHOSEN, AND WHY

LightGBM + Platt calibration, mirroring Static/Memory exactly (split → calibrate → derive threshold from our own data → ONNX export with calibration merged). One deliberate hyperparameter deviation from Memory's config: `is_unbalance=True` , because Network's train split is 84.3% benign / 15.7% malicious (~5.4:1), unlike Memory's near-50/50 — blindly copying Memory's omission would have reused a choice made for a different class balance. `num_leaves=31` , `max_depth=-1` , `learning_rate=0.05` , `min_child_samples=20` . 760 boosting iterations (early stop at 760/810), 54.2 s fit, peak RSS 2.8 GiB.

Divergence from the reference: deliberately **not** the autoencoder + classifier hybrid some network anomaly-detection designs use — that is a legitimate future capability, not a starting point. A single-feature-AUC sweep (the same check that caught Memory's shortcut) found **0 of 78 features exceed 0.95 AUC** on train or test; top feature `Fwd Seg Size Min` (train AUC 0.736 / test 0.848) is informative, not a giveaway — this model's performance reflects genuine multi-feature learning despite CSE-CIC-IDS2018 also being a fixed-testbed capture.

DATA-QUALITY HANDLING (10 ISSUES, EACH HANDLED DELIBERATELY)

Non-uniform column count (the `Thursday-20-02-2018` file has 84 vs 80 — 4 leading identity columns **dropped**, not aligned, so a model cannot memorise which fixed testbed machine was the attacker); `Timestamp` excluded from features (each day is one attack scenario — a near-perfect label proxy); 59 literal header-repeat rows filtered via an allowlist of real observed labels; `Flow Byts/s` / `Flow Pkts/s` NaN/Inf rows dropped, not imputed; inconsistent label taxonomy sidestepped via case-insensitive `!= "benign"` ; two-phase memory-safe loading capped the working set at 2,000,000 rows (peak RSS 22.8 GiB, ~8.5 min). **495,181 / 2,000,000 rows (24.75%) were exact duplicates** — union-merged into atomic groups before splitting. **4,742 duplicate groups spanned both benign and malicious labels**

(identical 78-feature vector, contradictory ground truth — a CICFlowMeter representation limit, not leakage): **excluded entirely from train, val, and test** (48,329 rows, 2.42%), saved separately for inspection. A real `stratified_group_split` index bug (writing into a plain array using non-contiguous DataFrame index values) was found and fixed with an explicit `reset_index(drop=True)`.

SPLIT

Split	Rows	% (target)	Benign	Malicious
train	1,526,757	78.23% (80%)	1,286,894	239,863
val	211,697	10.85% (10%)	160,864	50,833
test	213,217	10.92% (10%)	160,869	52,348

Total 1,951,671 rows (2,000,000 minus the 48,329 excluded ambiguous rows). **Invariants verified:** 0 duplicate-feature-vector groups span both labels (post-exclusion); 0 duplicate feature-vector hashes span more than one split (1,591,298 unique vectors / 1,951,671 rows). A direct, accepted consequence of the leakage guarantee: FTP-BruteForce and DoS-SlowHTTPTest have severe train-support skew (1,007 and 1,302 train rows vs thousands in val/test) because their 18,376-row duplicate "degenerate flow" group had to land almost entirely in val/test.

THRESHOLD DERIVATION

`NETWORK_MALICIOUS_MIN = 0.9441855970306654`, via `find_threshold_for_fpr()` at **target FPR 0.001** against calibrated val+test-combined probabilities (424,914 rows, **321,733 benign** — ~0.0003% resolution per false positive, so sub-0.1% targets are defensible here, unlike Memory or Behavioral).

CURRENT PERFORMANCE

Held-out test: **AUC-ROC 0.9972, AUC-PR 0.9951, precision 0.9969, recall 0.9752, F1 0.9859, FPR 0.098%**. Per-attack-type detection is 99.9–100% for Bot, all DDoS/DoS variants, and SSH-BruteForce; 75–91% for the under-trained web brute-force classes; and **10.75% for Infiltration** (n_{test} 1,433, n_{train} 11,463). **ONNX export:** mean abs error $2.1\text{e-}4$, **max abs error 0.5465** — three orders looser than Static's/Memory's, root-caused to the ONNX TreeEnsemble spec (LightGBM stores split thresholds as float64; ONNX's tree operator is float32-only), made visible here because Flow Duration/IAT features run up to $\sim 1.15\text{e}8$. Measured practical impact: **only 3 of 213,217 predictions (0.0014%) flip** at the operating threshold (two are exact boundary ties; the third is actually more correct in ONNX); aggregate metrics shift only in the 4th decimal. Shipped as the final artifact with the limitation documented.

KNOWN LIMITATION — THE TWO REASONS FOR THE ALERT CAP

(a) Infiltration blind spot: 10.75% detection is not under-training (every other under-10,000-train class scored $\geq 91\%$ except the two duplicate-signature classes). Infiltration attacks are slow / low-volume and do not produce the flow-statistics signature CICFlowMeter captures well; a comparative study found infiltration on this

dataset is only well-detected via NetFlow-derived features. Deliberately not chased by lowering the threshold (that would spike FPR across every well-detected class). **(b) External-validation collapse:** models reported "nearly perfect" on CSE-CIC-IDS2018 — which this one's numbers are — have been independently shown to degrade toward random on external/real-world traffic. Plus a dataset-level ~7.5% label-noise ceiling (automatic time-window labeling, not per-flow human verification) this pipeline cannot detect or correct from inside.

3.5 Cortex-Emulation

IN PLAIN TERMS

Emulation analysis runs a sample inside **Speakeasy**, a lightweight Windows emulator, and records the API calls it makes — a way to observe behavior without letting the sample touch a real machine. A neural network then judges that trace. Testing showed a serious problem: a model trained on samples collected in January 2022 detects only about 42% of malware collected just three months later, and making the model smaller does not help. It also barely beats a trivial lookup table. Because of this, **Emulation is trained but kept entirely out of the decision** — its verdict is logged for analysis only.

WHAT IT DETECTS AND WHY IT IS A SEPARATE SIGNAL

Malicious behavior observed in a Speakeasy emulation trace (API-name sequence), before the sample runs on a real host — complementary to Behavioral's real-execution traces.

DATASET & PROVENANCE

Quo Vadis (Trizna et al., 2022/2024), HuggingFace `dtrizna/quovadis-speakeasy` — Speakeasy-emulator-generated malware/benign execution reports. **Licence verified explicitly before downloading** (Apache-2.0, confirmed via both the HF `license` tag and the card text). 92,700 files: **75,298 train collected January 2022, 17,402 test collected April 2022** — a deliberate concept-drift split by the dataset's authors, **preserved, not re-randomised**. ~5.5 GiB.

ARCHITECTURE CHOSEN, AND WHY

Same family as Cortex-Behavioral — 1D-CNN + multi-head self-attention (`models/emulation_cnn.py`) — with three deliberate, documented deviations tuned to this data: `embed_dim=64` (not Behavioral's 128, because the vocabulary is ~8× larger — 3,152 real API names — with no more training data, so a bigger embedding table would add untrainable dimensions); `dropout=0.4` (not 0.3, to resist memorising rare-token patterns); `batch_size=64` ; `MAX_SEQ_LEN=500` (p90–p99 of the real sequence-length distribution). Conv/attention capacity kept unchanged as a reasoned starting point (Behavioral's own train set, ~7,340 examples, is the same order of magnitude as this signal's 6,070). 777,345 parameters. Raw sigmoid, no calibration layer (checked: the Platt-calibration bug fixed for Static's ONNX export was LightGBM- `predict_proba` -specific with no torch-path analogue).

Divergence from the reference: deliberately **not** the reference GRU / opcode design — the real data has no opcode or memory-access-pattern signal. Modeling population restricted to `module_entry` rows only; `thread / tls_callback_*` rows stay in the canonical parquet but are excluded from train/val/test. Within `module_entry`, exact-duplicate API-name sequences are collapsed to one representative per unique sequence (a `duplicate_count` column is retained), done **separately per era** so the concept-drift comparison is preserved.

SPLIT & TRAINING

Train 6,070 unique sequences (benign 4,044 / malicious 2,026), val 675 (450/225), test 2,495 (1,936/559). Early stop on val ROC-AUC plus an explicit train/val AUC-gap overfitting monitor (stop if `train_auc - val_auc > 0.05` for 3 consecutive epochs). The primary run (`embed_dim=64`) hit the overfitting stop at epoch 22 (best epoch 19, val AUC 0.9495); val AUC plateaued ~0.94 by epoch 8 while train AUC ran to 0.995 — the model memorises the ~6k collapsed training sequences trivially. Held-out test AUC-ROC **0.8737**.

THRESHOLD DERIVATION

`EMULATION_MALICIOUS_MIN = 0.999358594417572` — the target-FPR-1% point from the val+test sweep (2,386 benign, **pooling Jan and Apr eras, flagged explicitly**). **For logging / telemetry only**. The score distribution is strongly bimodal; at this threshold pooled detection is only **~49.9%** (~24 expected FP). The 0.1% target (~2 FP) is too thin to trust.

CURRENT PERFORMANCE AND THE TWO DISQUALIFYING FINDINGS

Finding 1 — ablation-confirmed Jan→Apr concept-drift collapse. At the 1%-FPR threshold, malicious recall drops from **70.7% (Jan-era slice) to 41.5% (Apr-era slice)**, AUC-ROC 0.949 → 0.874; benign FPR holds ~0.7–1% in both eras, so the loss is entirely malicious-side. Re-running at `embed_dim=32` (the overfitting stop's own first recommendation): the stop fired earlier (epoch 18, worse on every axis) and the drift gap **did not move** — recall 68.0% → 38.3% (29.7 pp gap vs 29.2 pp at `embed_dim=64`). **Capacity reduction fixed nothing** — this is a genuine train/test distribution shift, not an overfitting artifact. A per-family diagnostic shows the model memorises token sequences rather than learning generalizable behavior: it fires near what it memorised and goes quiet on modest drift, and for a family with too thin a distinct-sequence base (`coinminer` : 185 unique train sequences vs `ransomware` 's 1,065) it never builds a firing region at all — `coinminer` stays ~15–25% detected across every match bucket, including exact train matches.

Finding 2 — thin margin over a trivial baseline. Test accuracy at the 1%-FPR threshold is **86.4%** (`embed_dim=64`) / 85.6% (`embed_dim=32`), against a majority-class baseline of **77.6%** and an exact train-sequence duplicate-lookup of **82.2%**. The model beats the lookup table by only **+4.1 pp** / +3.3 pp.

DISPOSITION — STRICTER THAN AN ALERT CAP

Cortex-Emulation has **no branch in** `decide()`. `emulation_verdict_from_score()` and `EmulationVerdict` exist for logging only. This mirrors the earlier research repo's own Behavioral v2 category signal (`deployment_status: additive_report_only_not_in_policy_decision`). A signal that recovers ~40% of malware on the next collection era and barely beats a lookup table cannot carry autonomous or

ALERT authority. No ONNX export and no `pipeline.py` wiring for this signal. A genuine fix would most likely need engineered behavioural-category features (coarser file/registry/network/injection activity classes, hypothesised to be more temporally stable) or training data spanning more collection dates — explicitly **not** more hyperparameter tuning, which the ablation ruled out.

Policy Engine / Decision Logic

IN PLAIN TERMS

The policy engine turns up to five signal verdicts into **one** final decision. It does this with a strict checklist, top to bottom: the first rule that matches wins. It **never averages or blends scores** — a weak "maybe" from four signals cannot add up to a strong verdict. The order encodes trust: a Static BLOCK is checked first and is always final; a Behavioral MALICIOUS comes next and can TERMINATE a process; Memory and Network can only ever raise an ALERT, no matter how confident they are; Emulation is not consulted at all. Any error or failed check falls through to ALERT — never to a silent "allow".

THE PRIORITY CHAIN (INFERENCE/POLICY_ENGINE.PY :: DECIDE())

#	Condition	Final decision	Reason code
1	static verdict == BLOCK	BLOCK	static_block
2	behavioral verdict == MALICIOUS	TERMINATE	behavioral_malicious
3	memory verdict == MALICIOUS	ALERT (capped)	memory_malicious
4	network verdict == MALICIOUS	ALERT (capped)	network_malicious
5	static verdict == ALERT	ALERT	static_alert
6	static verdict == ERROR	ALERT	static_scan_error
7	behavioral verdict == ERROR	ALERT	behavioral_scan_error
8	memory verdict == ERROR	ALERT	memory_scan_error
9	network verdict == ERROR	ALERT	network_scan_error
10	otherwise	ALLOW	no_malicious_evidence

Emulation appears nowhere in this chain by design. There is no `emulation_verdict ==` branch.

AUTONOMOUS AUTHORITY VS. CAPPED AUTHORITY — AND EXACTLY WHY EACH CAP EXISTS

Signal	Max action it can cause	Capped?	Why the cap / why not
Static	BLOCK	No cap. Checked first, unconditional.	The one verdict in the whole system with no cap and no dependency on any other signal. Validated across three independent implementations (AUC 0.997-0.999) and against real malware and signed Microsoft tools.
Behavioral	TERMINATE	No cap. Second in priority.	Strong AUC (0.992-0.997) across three implementations; a MALICIOUS runtime-behavior

Signal	Max action it can cause	Capped?	Why the cap / why not
			verdict is high-confidence evidence of an active threat and warrants killing the process.
Memory	ALERT only	Capped. Cannot TERMINATE/ BLOCK regardless of score.	Trained/threshold-derived on CIC-MalMem-2022 only (a fixed lab corpus), not validated against real injected-process samples. Its near-perfect test metrics are themselves suspect: 22/62 features individually exceed 0.95 AUC, consistent with the dataset's single-VM benign collection. Letting an unvalidated signal autonomously kill a process risks terminating a legitimate one on a false positive with no field evidence to rule it out.
Network	ALERT only	Capped. Same rung as Memory.	Two independent legs: (a) Infiltration detection is only 10.75% — Network structurally cannot be trusted alone for slow/low-volume attacks, exactly the class Behavioral and Memory have process-level visibility into; (b) a documented external-validation-collapse pattern for CSE-CIC-IDS2018 (near-perfect lab scores degrade toward random on real traffic), plus a ~7.5% dataset label-noise ceiling. Having both legs is why the cap is treated as at least as firm as Memory's.
Emulation	None — report-only	Not in <code>decide()</code> at all.	Ablation-confirmed Jan→Apr 2022 temporal concept-drift collapse (malicious recall ~71% → ~42%, unchanged by capacity reduction) and only ~+3-4 pp test accuracy over a trivial train-sequence duplicate-lookup. A signal that loses ~40% of its malware detection on the next collection era, and barely beats a lookup table, cannot carry autonomous or ALERT authority. Revisit only with a materially different model.

DESIGN INVARIANTS

- **Verdicts, not scores.** `decide()` receives enum verdicts; the numeric scores never enter it. No weighting, averaging, or ensembling anywhere in the decision path.
- **Static BLOCK is always protected.** Rule 1 returns immediately; nothing downstream can soften it, and Behavioral is never even run after a BLOCK.
- **Upgrade-only for lower signals.** Memory/Network MALICIOUS can raise an otherwise-ALLOW/ALERT outcome to ALERT, but can never reach BLOCK or TERMINATE.
- **Fail-closed.** Every ERROR path yields ALERT. The default final decision on the dataclass is ALERT, not ALLOW; ALLOW is only reached by explicitly exhausting every malicious/error condition.

SECTION 5

Known Limitations and Risk Mitigations

This section is deliberately prominent, not an appendix. Every limitation below is real, documented in the repository, and reflected in how much authority the policy engine grants the affected signal. For each: the risk in plain language, the technical evidence, why the policy engine caps that signal as a direct, evidence-based consequence, and what it would take to lift the cap.

5.1 Memory — single-VM benign artifact

THE RISK, PLAINLY

The Memory model's benign training examples all come from **one baseline virtual machine**. The model may have learned to recognise "that specific machine's normal state" rather than "the absence of malicious activity." On a different clean machine it could raise false alarms — or, worse, miss malice on a system that simply does not look like the training VM.

Technical evidence. A single-feature-AUC sweep over all 62 raw+derived features found **22 of them individually exceed 0.95 AUC**, confirmed identically on the training split and the never-touched test split (ruling out our own split leaking — separately verified: 0/32,106 sample-id groups and 0/58,596 duplicate feature-vector hashes span splits). Example bands (train): `handles.avg_handles_per_proc` 208–318 for benign (std 17.5) vs 71–33,784 for malware; `dlllist.avg_dlls_per_proc` 34.5–53.2 for benign vs down to 7.3 for malware. One derived feature (`svcsan_driver_ratio`) has 5.8× the gain of the next-best. This pervasive, held-out-stable separability matches CIC-MalMem-2022's documented collection methodology: every benign sample is a repeated capture of "normal user behavior" on a single baseline Windows 10 VM; malicious samples span far more varied executions.

Why the policy engine caps Memory at ALERT. Because the near-perfect held-out metrics (AUC 1.0, recall 1.0) are plausibly an artifact of that single-VM benign collection, they are evidence to keep the cap, not to lift it. An unvalidated signal must not autonomously TERMINATE/BLOCK a process it cannot be trusted to have judged on malice rather than machine identity. Memory still runs as a fully independent signal (not gated behind Static) so it can raise an ALERT on fileless activity.

What would lift the cap. Memory captures from **multiple genuinely different, non-baseline benign machines**, plus validation against real injected-process samples outside CIC-MalMem-2022's test set. More rows from the same dataset would not qualify. Also open: a way to identify benign near-duplicate capture sessions (only exact duplicates are currently merged).

5.2 Network — Infiltration blind spot & external-validation collapse

THE RISK, PLAINLY

Two things. First, the Network model is **almost blind to slow, quiet intrusions** ("Infiltration"): it catches only about one in ten. Second, models that look near-perfect on this particular dataset have repeatedly been shown to **fall apart on real-world traffic**. So its clean lab numbers should not be read as production-ready.

Technical evidence — Infiltration. Held-out test detection for Infiltration is **10.75%** ($n_{\text{test}} 1,433$) despite **11,463 training rows** — every other class with under 10,000 train rows still scored $\geq 91\%$ (except FTP-BruteForce and DoS-SlowHTTPTest, which are high only because of a near-degenerate duplicate flow signature). This is a CICFlowMeter feature-representation ceiling: infiltration is slow / low-volume by design and does not produce the flow-statistics signature CICFlowMeter captures well; an independent comparative study found infiltration on this dataset is only well-detected using NetFlow-derived features. Lowering the threshold to chase it would spike FPR across every well-detected class.

Technical evidence — external validation. A documented pattern specific to CSE-CIC-IDS2018: models reported "nearly perfect" on it (Cortex-Network's test AUC is 0.9972) have been independently shown elsewhere to degrade toward random performance on external/real traffic. Plus an independently reported **~7.5% label-noise ceiling** from the dataset's automatic time-window-based labeling — some fraction of both the training signal and the "ground truth" behind every metric here is simply wrong, undetectably from inside the dataset.

Why the policy engine caps Network at ALERT. Two independent legs, either sufficient alone: (a) the Infiltration gap means Network cannot be trusted alone for slow/quiet threats — but that class is exactly what Behavioral (API sequences) and Memory (process/injection artifacts) have process-level visibility into, so system coverage still holds without giving Network autonomous authority; (b) external-validation collapse means the clean test numbers cannot be assumed to predict field performance. Having both legs is why this cap is treated as at least as firm as Memory's.

What would lift the cap. Independent validation on real network traffic that CSE-CIC-IDS2018 did not generate; and, for Infiltration specifically, a NetFlow-derived feature representation (a documented future option, not applied).

5.3 Emulation — confirmed Jan→Apr concept-drift collapse

THE RISK, PLAINLY

The Emulation model was trained on samples from January 2022 and tested on samples from April 2022 — only three months apart. Its malware detection **fell by roughly 30 percentage points** over that gap. Shrinking the model to reduce overfitting did not change this at all, which means the model is memorising specific call sequences rather than learning what malicious behavior looks like. It also barely beats a lookup table.

Technical evidence. At the 1%-FPR threshold, malicious recall drops from **70.7% on the Jan-era slice to 41.5% on the Apr-era slice** (AUC-ROC 0.949 → 0.874); benign FPR holds ~0.7–1% across both eras, so the loss is entirely malicious-side. A pre-committed capacity

fallback was tested: at `embed_dim=32` the overfitting stop fired earlier (epoch 18 vs 22, worse on every axis) and the drift gap **did not move** — 68.0% → 38.3% (29.7 pp) vs 29.2 pp at `embed_dim=64`. Per-family diagnosis: `coinminer` (185 unique train sequences) stays ~15–25% detected in every match bucket including exact train matches — the model never built a firing region for it — while `ransomware` (1,065 unique train sequences) tracks match quality (exact-match Apr rows 70% detected, near-match only 29%). Mechanism: **the model memorises token sequences rather than learning generalizable behavior**. Margin check: test accuracy 86.4% vs a majority-class 77.6% and an exact train-sequence duplicate-lookup 82.2% — **+4.1 pp over the lookup table**.

Why the policy engine excludes Emulation entirely. This is stricter than the Memory/Network ALERT cap: Emulation has **no branch in `decide()`** and its threshold is set for telemetry only. A signal that loses ~40% of its malware detection on the very next collection era, with capacity reduction ruled out as the lever, and that beats a lookup table by ~3–4 pp, cannot carry autonomous or ALERT authority. This mirrors the earlier research repo's own precedent for its Behavioral v2 category signal.

What would lift the report-only status. A materially different model, not more tuning: most likely engineered behavioural-category features (coarser activity classes, hypothesised to be more temporally stable than exact API-call token sequences) or training data spanning more collection dates. Then re-assess authority from scratch.

5.4 Static — the recently-found, partially-fixed feature-extraction gap

THE RISK, PLAINLY

A broken optional import meant one whole group of features (the digital-signature summary) was being filled with zeros for **every file scanned live** — while the training data had real values there. That mismatch quietly pushed live scores toward "malicious." The import is now fixed, but the live feature extractor still does not perfectly match the reference used to build the training data, and there is **no automated test** that would catch the next such regression.

Technical evidence. `features/pe_features.py` imported from `signify.authenticode` import `SignedPEFile`, an API removed in the installed **signify 0.9.2** (now `AuthenticodeFile`). The module's `except ImportError` swallowed the failure, `_SIGNIFY_AVAILABLE` became `False`, and the 8-dim `authenticode` group returned **all-zeros for every file at inference** — a train/serve skew, because EMBER2024's training vectors carry real authenticode features. It was caught only by manually re-scanning the 5 cortex-endpoint validation files and noticing `svchost.exe` (a signed Microsoft binary) scored 0.99 where cortex-endpoint scored 0.47.

Partial fix applied: ported to the 0.9.x API; `svchost.exe` now parses a genuine chain (`num_certs=1`, real countersigning timestamp), and the `authenticode` group contribution moved from +1.17 (wrongly toward malicious) to -1.33 (toward benign), shifting its score 0.9913 → 0.9839. **Not fixed:** `svchost.exe` still does not reach ALLOW (0.9839 vs 0.4744). The residual ~2.0 raw-margin gap is a broader fidelity problem — `pe_features.py` produces a vector systematically shifted toward "malicious" vs the thrember reference (`header` pushes

harder: +3.75 vs +3.36; benign-side groups softer: `histogram` -0.51 vs -0.83; the hand-rolled 8-feature `authenticcode` group carries less signal than thrember's even when populated: -1.33 vs -2.12).

Current 5-file comparison (post-fix, static-only — behavioral/memory have no input, network isn't wired):

File	cortex-endpoint	cortex-ml (post-fix)	Agree?
<code>svchost.exe</code>	0.4744 / allow	0.9839 / ALERT	No — feature-fidelity gap
<code>notepad_test.exe</code>	0.0057 / allow	0.0062 / ALLOW	Yes
<code>benign_test_50mb.exe</code>	0.9690 / alert	0.9950 / BLOCK	No — stricter BLOCK threshold + ~3e-5 boundary tie
<code>notepad.exe</code>	0.7746 / alert	0.9917 / ALERT	Yes (verdict)
<code>extractor.exe</code>	0.9820 / block	0.9956 / block	Yes

Why this matters for policy. Static holds the highest, uncapped authority in the system, so a silent extractor regression degrades every live static score — including BLOCK decisions — with no error surfaced. The gap does not currently change Static's authority tier (it remains validated across three implementations), but it is the **top open correctness item** for the static path and a live-score accuracy risk until closed.

What would close it. (1) Build an automated feature-parity test comparing `pe_features.py` output against a pinned thrember reference on real PEs including a signed one — the check that would have caught the signify regression immediately (cortex-endpoint has exactly this; `cortex-ml` does not). (2) Close the residual per-group fidelity gap so the live vector stops skewing toward "malicious."

SECTION 6

Engineering Discipline

IN PLAIN TERMS

Every one of the five signals was built the same careful way: check the data's structure against the **actual files**, not the documentation (which was wrong more than once); actively hunt for **data leakage** — where the answer accidentally leaks from the training set into the test set — rather than assume the data is clean; and **re-derive every decision threshold from scratch** against the model actually trained here, never copy a number from a previous project. This process caught real, score-changing bugs in every signal.

SCHEMA VALIDATION AGAINST REAL DATA, NOT DOCUMENTATION

- **Memory:** CIC-MalMem-2022 validated structurally against the real file — column count, dtypes, exact `class` value set, first/last column identity.
- **Emulation:** the Quo Vadis dataset card's documented schema is wrong — it describes each file as `{"sha256": ..., "entry_points": [...]}`; the real files are a bare JSON list with no wrapper and no `sha256` field (recoverable only from the filename). One category (`report_windows_syswow64`, 294 files) uses a completely different filename scheme. The card's own row-count table is off by 3 from the real repository; its prose numbers are staler still. All verified by a full diff against the real file listing.
- **Network:** the real S3 bucket structure and the 84-vs-80 column discrepancy were confirmed against the actual CSVs, not assumed from the docs; the label allowlist was built by scanning 16.2M real label values.

REAL LEAKAGE CHECKS — WITH THE BUGS THEY CAUGHT

- **Behavioral:** a naive split left **85 sequences spanning multiple splits**; fixed with a grouped stratified split keyed on identical `api_calls` sequences (verified 0 after).
- **Memory:** a grouping-detection bug (using only "median group size > 1") treated the constant "Benign" Category as one giant real group and put all synthetic benign rows in one split — caught on a synthetic dataset shaped like the real one before it ever ran on real data. Then, on the real 58,596 rows, **20 duplicate feature-vector hashes (80 rows) spanned multiple splits**; fixed with a union-find merge that unions sample-id groups with exact-duplicate-vector groups before allocation, making cross-split duplication structurally impossible.
- **Network:** **495,181 / 2,000,000 rows (24.75%) exact duplicates; 4,742 duplicate groups spanning both benign and malicious labels** excluded entirely from train/val/test; and a real `stratified_group_split` index bug (writing into a plain array with non-contiguous index values, "working by coincidence") fixed with `reset_index(drop=True)`.
- **Emulation:** **96.8% of entry-point rows were exact duplicates**; an early naive split produced a val set that was 90.2% malicious because one 4,312-row duplicate group dominated it. The `apihash` field was validated rather than trusted — **686 / 6,699**

apihash groups (10.2%) were false merges — so the pipeline falls back to a hand-computed sequence hash.

THRESHOLDS RE-DERIVED FROM SCRATCH, NEVER INHERITED

Every signal's operating threshold is derived against its own trained model's calibrated probabilities — Static (`find_threshold_for_fpr()` at FPR 0.01 / 0.001 on 539,940 test rows), Behavioral (a sweep on val+test, deliberately not a precise FPR target given only 274 benign), Memory (FPR 0.01 on 5,860 benign), Network (FPR 0.001 on 321,733 benign), Emulation (FPR 1% sweep, telemetry only). None carried over the prior project's placeholder values. The README states plainly: "If you retrain any model from scratch, the raw score distribution will differ and these need to be re-derived again, not assumed to still hold."

THREE CONCRETE BUGS FOUND AND FIXED — EVIDENCE OF PROCESS QUALITY

1 • The Platt-calibration bug (Static ONNX export). `export_static_lgbm_to_onnx()` had been written earlier but never run against the real trained model. First run surfaced that the raw LightGBM booster's output and `predict_proba()` (which applies a Platt calibrator on top) **differ by up to 0.213** on real test data — and the deployed thresholds were derived against the calibrated distribution. Exporting the bare booster would have silently shipped a model whose scores no longer matched its thresholds. Fixed by merging the calibrator's `LogisticRegression` (via `skl2onnx`) into the same ONNX graph as the booster (via `onnxmltools`), chained after a `slice` that extracts the malicious-class column. Two further version bugs fixed en route (`opset-15` cap hardcoded to 17; IR-version / duplicate-tensor-name mismatches before `merge_models`). Verified against 10,000 real rows: max abs diff 0.0032, zero verdict differences.

2 • The Memory duplicate-hash leak. On the real 58,596-row dataset, **20 duplicate feature-vector hashes (80 rows, 0.14%)** spanned multiple splits — 8 groups / 55 rows benign-only (no per-sample identifier exists for benign rows), 11 groups / 25 rows cross-malware-family collisions (different samples, coincidentally identical coarse VolMemlyzer summary stats). A separate crash was also fixed:

`verify_zero_duplicate_feature_hashes` tried to hash the string `split` column itself, failing to cast "train" / "val" / "test" to float64. Final verified state: 0/32,106 sample-id groups and 0/58,596 duplicate feature-vector hashes span more than one split.

3 • The signify import break (Static live extraction). `features/pe_features.py` imported from `signify.authenticode` import `SignedPEFile`, removed in signify 0.9.2. The except `ImportError` swallowed it silently, so the 8-dim `authenticode` feature group returned all-zeros for every live scan while training vectors had real values — a train/serve skew invisible to offline eval. Caught only by a manual 5-file comparison (`svchost.exe` scoring 0.99 vs cortex-endpoint's 0.47). Now ported to the 0.9.x `AuthenticodeFile` API. This is precisely the class of regression an automated feature-

parity test would catch immediately — building one is the top open static-path item (\$5.4, §7).

SECTION 7

Current Status and Next Steps

IN PLAIN TERMS

Four of the five signals are fully trained, verified, and exported; the fifth (Emulation) is trained but intentionally parked. The core detection engine works end-to-end for the file-and-behavior path. The main gaps are validation-related: proving the newer signals on real-world data, adding a test that guards the static feature extractor, stress-testing the behavioral threshold, and packaging the whole thing as a runnable tool rather than a Python library.

WHAT IS TRAINED AND COMMITTED

Item	State	Git status
Cortex-Static — LightGBM, Platt-calibrated, ONNX-exported	Trained & verified (AUC 0.9988)	Committed (2026-08-19, 4 commits <code>6cd4131 ... 0e295c7</code>)
Cortex-Behavioral — 1D-CNN+attention, ONNX + INT8	Trained & verified (AUC 0.9973)	Committed (2026-08-19)
Cortex-Memory — LightGBM, ONNX-exported	Trained & verified (AUC 1.0)	Committed (2026-08-27, <code>9331159</code>)
Cortex-Network — LightGBM, ONNX-exported (documented precision caveat)	Trained & verified (AUC 0.9972)	Committed (2026-08-27, <code>9331159</code>)
Cortex-Emulation — 1D-CNN+attention	Trained; report-only; no ONNX; not wired	Committed (2026-08-27, <code>9331159</code>)
Policy engine + pipeline (Static→Behavioral gate, independent Memory)	Working end-to-end	Committed
<code>signify</code> 0.9.x fix + these documentation updates	Applied	Uncommitted working-tree change on top of <code>9331159</code>

WHAT IS STILL OPEN — THE ROADMAP, HONESTLY FRAMED

- 1. Real-world malware / signed-binary validation (in progress).** The 5 real test files have been run through `CortexPipeline.scan()` (2026-08-27): the pipeline works end-to-end (static-only for bare `.exe` s), verdicts agree on 3/5. `svchost.exe` disagrees (feature-fidelity gap, §5.4); `benign_test_50mb.exe` disagrees for a separate, expected reason (stricter BLOCK threshold + a $\sim 3e-5$ boundary tie). Broader validation against live malware and more signed binaries is not yet done.
- 2. The missing feature-parity test (top static-path item).** Nothing compares `pe_features.py` output against a reference (thrember) extractor on real PEs, signed one included — the check cortex-endpoint has and cortex-ml lacks. A silent extraction regression currently degrades every live static score with no error.

3. **Close the residual `pe_features.py` -vs-thrember fidelity gap** — the live vector skews toward "malicious" (`header` pushes harder, benign-side groups softer, hand-rolled `authenticode` weaker).
4. **Behavioral threshold generalization stress test.** Subject `cortex-ml` 's 0.60 threshold to a held-out generalization check structured like the one that broke the prior research-repo model (near-zero validation FPR → 3.09% test FPR).
5. **Memory & Network authority caps — flagged for revisit, not permanent.** Lifting them requires real-world validation beyond each dataset's own test split (real injected-process samples for Memory; real external traffic for Network), not better lab metrics.
6. **Emulation — report-only pending a materially different model.** Prototype engineered behavioural-category features or gather training data across more collection dates, then re-assess authority. No ONNX export until the model is worth shipping.
7. **Network pipeline wiring.** The model, threshold, and verdict function exist; `pipeline.py` needs a network-input path and a call into `decide()`.
8. **CLI packaging.** `cortex-ml` is a Python library, not yet a deployable agent — no packaged CLI (mirroring cortex-endpoint's `scan` / `scan-behavioral` / `scan-combined` surface) and no PyInstaller build. This gap has grown as signals were added. Prior lessons to reapply: exclude torch from the start, watch the matplotlib/PIL transitive dependency, budget for the self-flagging false-positive risk from day one.

Appendix — Full Metrics, Thresholds, Splits, Config

A. Exact threshold values (`inference/policy_engine.py` / `config/thresholds.yaml`)

Constant	Value	Rule	Derivation	Authority
<code>STATIC_ALLOW_MAX</code>	0.6163460957	score < this → ALLOW	<code>find_threshold_for_fpr()</code> , target FPR 0.01 (actual 0.009999, det. 0.9803), calibrated EMBER2024 test (539,940 rows)	BLOCK (uncapped)
<code>STATIC_BLOCK_MIN</code>	0.9950119117	score ≥ this → BLOCK (else ALERT)	same, target FPR 0.001 (actual 0.000997, det. 0.9168)	
<code>BEHAVIORAL_MALICIOUS_MIN</code>	0.60	score ≥ this → MALICIOUS	sweep 0.10–0.99 on val+test combined (1,835 rows, 274 benign); deliberately not raised to ~0.922 to clear 1 hard benign FP scoring 0.921006	TERMINATE
<code>MEMORY_MALICIOUS_MIN</code>	0.0005358335957155212	score ≥ this → MALICIOUS	<code>find_threshold_for_fpr()</code> , target FPR 0.01 on calibrated val+test combined (11,860 rows, 5,860 benign)	ALERT (capped)
<code>NETWORK_MALICIOUS_MIN</code>	0.9441855970306654	score ≥ this → MALICIOUS	<code>find_threshold_for_fpr()</code> , target FPR 0.001 on calibrated val+test combined (424,914 rows, 321,733 benign)	ALERT (capped; not wired)
<code>EMULATION_MALICIOUS_MIN</code>	0.999358594417572	score ≥ this → MALICIOUS (telemetry)	target-FPR-1% point from val+test sweep (2,386 benign, pools Jan+Apr eras)	Report-only
<code>sequence.max_api_calls</code>	100	Behavioral tokenizer fixed length	architecture constant	—
<code>file.max_size_bytes</code>	104,857,600 (100 MiB)	path-validation ceiling	pipeline constant	—

Full authority rationale for each tier is in §4. Emulation's constant exists for logging only — `decide()` has no emulation branch.

B. Per-signal held-out test metrics

Signal	Operating point	AUC-ROC	Precision	Recall / detection	FPR	Other
Static	threshold 0.5	0.9988	—	98.17%	1.10%	2,996 rounds, ~2.1 h
Behavioral	threshold 0.60	0.9973	100%	98.08%	—	acc 98.36%; 18 epochs, ~4 min CPU
Memory	threshold 5.36e-4	1.0	0.9891	1.0	1.13% (33/2,930)	F1 0.9945; best iter 71
Network	threshold 0.94419	0.9972	0.9969	0.9752	0.098%	AUC-PR 0.9951; F1 0.9859; 760 iters, 54.2 s
Emulation (e64)	1%-FPR threshold, pooled eras	0.8737	—	~49.9% pooled	~0.96%	acc 86.4%; best epoch 19; overfit-stop epoch 22

B.1 Memory threshold sweep (val+test combined, 11,860 rows, 5,860 benign)

Target FPR	Threshold	Actual FPR	Benign FPs	Detection
0.10%	0.100225	0.085%	5 / 5,860	99.95%
0.50%	0.000627	0.478%	28 / 5,860	100.00%
1.00% (chosen)	0.000536	0.939%	55 / 5,860	100.00%
2.00%	0.000502	1.724%	101 / 5,860	100.00%
5.00%	0.000446	4.744%	278 / 5,860	100.00%

B.2 Emulation threshold sweep (val+test combined, 2,386 benign, pools Jan+Apr eras)

Target FPR	Threshold	Actual FPR	Benign FPs	Detection
0.10%	0.999996	0.084%	2 / 2,386	4.2%
0.50%	0.999887	0.46%	11 / 2,386	44.3%
1.00% (primary)	0.999359	0.96%	23 / 2,386	49.9%
2.00%	0.960831	1.97%	47 / 2,386	55.0%
5.00%	0.676327	4.82%	115 / 2,386	67.1%

B.3 Emulation — concept-drift by era & per-family test recall (e64, 1%-FPR threshold)

Slice / family	n_train	n_test	Detection / recall	AUC-ROC
Jan-2022 era slice	—	—	70.7%	0.949

Slice / family	n_train	n_test	Detection / recall	AUC-ROC
Apr-2022 era slice	—	—	41.5%	0.874
Jan→Apr at embed_dim=32	—	—	68.0% → 38.3%	—
trojan	262	79	64.6%	—
backdoor	149	99	51.5%	—
ransomware	1,065	116	48.3%	—
dropper	193	44	47.7%	—
keylogger	112	64	29.7%	—
rat	60	69	27.5%	—
coinminer	185	88	17.1%	—
clean (benign FPR)	4,003	1,924	0.68%	—
windows_syswow64 (benign FPR)	41	12	0.00%	—

Baselines at the 1%-FPR threshold: model test accuracy 86.4% (e64) / 85.6% (e32) vs majority-class 77.6% vs exact train-sequence duplicate-lookup 82.2% (571/2,495 test sequences appear verbatim in train).

B.4 Network — per-attack-type test detection

Label	n_test	n_train	Detection	Read
Bot	3,517	27,882	99.94%	—
Brute Force -Web	64	484	75.00%	under-trained
Brute Force -XSS	23	183	91.30%	under-trained
DDOS attack-HOIC	8,402	67,211	100.00%	—
DDOS attack-LOIC-UDP	173	1,384	100.00%	—
DDoS attacks-LOIC-HTTP	7,056	56,453	100.00%	—
DoS attacks-GoldenEye	508	4,067	100.00%	—
DoS attacks-Hulk	5,657	45,255	100.00%	—
DoS attacks-SlowHTTPTest	9,105	1,302	100.00%	low train support, duplicate-signature effect
DoS attacks-Slowloris	1,099	8,792	99.91%	—
FTP-BruteForce	13,005	1,007	100.00%	low train support, duplicate-signature effect
Infiltration	1,433	11,463	10.75%	feature-representation ceiling, not under-training
SQL Injection	8	72	100.00%	tiny sample

Label	n_test	n_train	Detection	Read
SSH-Bruteforce	2,298	14,308	100.00%	—
Benign	160,869	1,286,894	FPR 0.10%	—

C. Dataset sizes & splits

Signal	Dataset (licence)	Raw	Train	Val	Test
Static	EMBER2024, PE-format (Apache-2.0)	4,680,000 train / 1,080,000 test raw ($\approx 2 \times$ dup)	2,340,000 unique (100% of dedup used)	200,000 (from fitting pool in reference pipeline)	539,940 unique
Behavioral	Mal-API-2019 + MalbehavD-V1 + Carpenter benign (GitHub / GitHub / Kaggle)	9,778 combined pre-dedup (7,107 + 2,570 + 101)	80% grouped-stratified by (source,label)	10% (val+test combined = 1,835 rows, 274 benign)	10%
Memory	CIC-MalMem-2022 (public research; Kaggle mirror used)	58,596 rows, 55 VolMemLyzer features	46,736 (benign 23,438 / malware 23,298)	5,930 (2,930 / 3,000)	5,930 (2,930 / 3,000)
Network	CSE-CIC-IDS2018 (explicit redistribution licence; AWS Open Data)	16,233,002 raw $\rightarrow$ 2,000,000 capped $\rightarrow$ 1,951,671 after excluding 48,329 ambiguous	1,526,757 (78.23%; benign 1,286,894 / mal 239,863)	211,697 (10.85%; 160,864 / 50,833)	213,217 (10.92%; 160,869 / 52,348)
Emulation	Quo Vadis / dtrizna/ quovadis-speakeasy (Apache-2.0, verified)	92,700 files (75,298 Jan-2022 train / 17,402 Apr-2022 test); module_entry rows only, exact-dup sequences collapsed per era	6,070 unique sequences (benign 4,044 / mal 2,026)	675 (450 / 225)	2,495 (1,936 / 559)

Leakage invariants verified: Static 0 train/test hash overlap; Behavioral 0 sequences span >1 split (85 fixed); Memory 0/32,106 sample-id groups and 0/58,596 duplicate feature-vector hashes span >1 split; Network 0 duplicate groups span both labels (post-exclusion) and 0 hashes span >1 split (1,591,298 unique vectors); Emulation every split 100% unique sequences, 0 train/val overlap, `duplicate_count` sums back to pre-collapse totals.

D. Model architecture & hyperparameter reference

Signal	Architecture	Key hyperparameters	Calibration	ONNX
Static	LightGBM GBDT over 2568-dim / 12-group EMBER2024-	<code>num_leaves=255</code> , <code>max_depth=15</code> , <code>learning_rate=0.03</code> ,	Platt (<code>LogisticRegression</code>),	61.4 MB; max abs diff 0.0032 on 10k rows; 0

Signal	Architecture	Key hyperparameters	Calibration	ONNX
	compatible feature vector	<code>min_child_samples=50</code> , <code>is_unbalance=True</code> , 2,996 rounds	merged into ONNX graph	verdict diffs; no INT8
Behavioral	Embedding + learned positional emb → conv stack 128→256→128→128 (k=3,5,7,3, BN+dropout 0.3) → 4-head self-attention (residual+LayerNorm) → global pool → sigmoid	seq len 100, embed_dim=128 (CLI default; 64 optional), AdamW, cosine restarts, early stop; 18 epochs	None — raw sigmoid	0.36 MB class (prior impl); INT8 restricted to MatMul/Gemm; legacy exporter forced
Memory	LightGBM GBDT over 55 raw VolMemLyzer + 7 derived ratio features (62 total)	<code>num_leaves=31</code> , <code>max_depth=-1</code> , <code>learning_rate=0.05</code> , <code>min_child_samples=20</code> , <code>is_unbalance</code> omitted (≈50/50), best iter 71	Platt, merged into ONNX graph	mean abs err 6.7e-9, max 2.0e-6 on 5,930 rows; 0 mismatches; no INT8
Network	LightGBM GBDT over 78 CICFlowMeter flow features (mirrors Static/Memory shape; not an autoencoder hybrid)	<code>num_leaves=31</code> , <code>max_depth=-1</code> , <code>learning_rate=0.05</code> , <code>min_child_samples=20</code> , <code>is_unbalance=True</code> (≈5.4:1), 760 iters	Platt, merged into ONNX graph	mean abs err 2.1e-4, max 0.5465 (float64→float32 TreeEnsemble spec limit); 3/213,217 flips at threshold; shipped as final
Emulation	Behavioral-family 1D-CNN + 4-head self-attention (<code>emulation_cnn.py</code>); not the reference GRU/opcode design	vocab 3,154 (train-only), seq len 500, <code>embed_dim=64</code> , <code>dropout=0.4</code> , <code>batch_size=64</code> , 777,345 params; val-AUC early stop + AUC-gap overfit monitor	None — raw sigmoid	None (deferred until the model is worth shipping)

Static feature groups (sum 2568): general 7, histogram 256, byteentropy 256, strings 177, header 74, section 224, imports 1282, exports 129, datadirectories 34, richheader 33, authenticcode 8, pefilewarnings 88.

E. Config & source references

- **Decision logic:** `inference/policy_engine.py` — `decide()` , verdict enums, threshold constants (with full derivation comments).
- **Scan pipeline:** `inference/pipeline.py` — `CortexPipeline.scan()` , `_validate_path()` , `_run_memory()` , `_run_behavioral()` , `to_security_event()` .
- **Thresholds mirror:** `config/thresholds.yaml` — `static` , `behavioral` , `memory` , `network` , `emulation` , `sequence` , `file` blocks.

- **Features:** `features/pe_features.py` (2568-dim, 12 groups), `features/memory_features.py` (7 derived + train-only scaler), `features/ember2024_adapter.py`.
- **Models:** `models/static_lgbm.py`, `models/memory_lgbm.py`, `models/network_lgbm.py`, `models/behavioral_cnn.py` + `train_behavioral.py`, `models/emulation_cnn.py` + `train_emulation.py`.
- **Data acquisition:** `data/download_ember2024.py`, `download_memory.py`, `download_network.py`, `download_emulation.py`, `download_behavioral.py`.
- **Splits:** `scripts/split_behavioral.py`, `split_memory.py`, `split_network.py`, `split_emulation.py`.
- **Export:** `export/export_onnx.py`, `export/quantize.py`.
- **Narrative sources:** `PROJECT_HISTORY_REPORT.md` (Era 3 + STEP 3), `README.md` (Cortex-Memory / Cortex-Network / Cortex-Emulation sections, "Known gap" and "Open items").

Report compiled 2026-08-27 from the `cortex-ml` repository at commit `9331159` plus the uncommitted `signify 0.9.x` fix and documentation updates. All metrics, thresholds, split counts, and limitations are quoted directly from `PROJECT_HISTORY_REPORT.md`, `README.md`, `inference/policy_engine.py`, `inference/pipeline.py`, and `config/thresholds.yaml`.